

TP3 – Intégration de la sauvegarde

4PYTH – Projet fil rouge « Fondation » · En autonomie, du 10 octobre au 5 novembre



Contexte

La galaxie est en plein chaos et la gestion de vos équipages devient critique. Aujourd'hui, chaque fois que le programme s'arrête, la flotte revient à son état de départ : recrues, promotions et vaisseaux ajoutés disparaissent.

Pour ne plus rien perdre, votre poste de commandement doit **sauvegarder** l'état de la flotte dans un fichier et savoir le **recharger**... sans se laisser piéger par un fichier abîmé ou falsifié.

Ce TP en bref

- **Objectifs** : convertir des objets en dictionnaires, sauvegarder et charger en JSON, gérer les erreurs de fichier et valider des données venues de l'extérieur.
- Il se fait **en autonomie**. Le projet complet est à rendre **au plus tard le jeudi 5 novembre à 16h30** : seul le dernier commit poussé avant cette heure est évalué.
- Commitez et poussez **régulièrement** : je suis l'avancée de vos dépôts pendant ces quatre semaines.
- Bloqué ? Relisez le cours 3, puis écrivez-moi : chris.chevalier@ecole-hexagone.com.
- L'IA est autorisée pour le projet, à une condition : **comprendre chaque ligne** de votre code. Je le vérifierai en soutenance.

1

Des objets aux dictionnaires

≈ 1 h

Méthodes · héritage · composition

JSON ne connaît pas vos classes : `json.dumps(fleet)` provoque une `TypeError`. Chaque classe doit donc savoir se **convertir en dictionnaire**, avec une méthode `to_dict()`.

Classe	<code>to_dict()</code> renvoie...
Person	<code>first_name</code> , <code>last_name</code> , <code>gender</code> et <code>age</code>
Operator	<code>"type": "operator"</code> , les clés de <code>Person</code> , puis <code>role</code> et <code>experience</code>
Mentalist	<code>"type": "mentalist"</code> , les clés de <code>Person</code> , puis <code>mana</code>
Spaceship	<code>name</code> , <code>ship_type</code> , <code>condition</code> et <code>crew</code> : la liste des <code>to_dict()</code> de chaque membre
Fleet	<code>name</code> et <code>spaceships</code> : la liste des <code>to_dict()</code> de chaque vaisseau

Le bon format, vous l'avez déjà

Le résultat de `fleet.to_dict()` a **exactement le format de `FLEET_DATA`** du TP2, avec une clé `experience` en plus pour les opérateurs. Votre `build_fleet()` saura donc reconstruire la flotte : adaptez-le seulement pour lire l'expérience avec `m.get("experience", 0)`, puisqu'elle est absente de `FLEET_DATA`.

Vérification – l'aller-retour doit être parfait

```
fleet = build_fleet(FLEET_DATA)
fleet.spaceships[0].crew[0].act()           # Hugo Crast pilote le vaisseau.

data = fleet.to_dict()
print(data["spaceships"][0]["crew"][0])
# {'type': 'operator', 'first_name': 'Hugo', 'last_name': 'Crast', 'gender': 'M',
#   'age': 37, 'role': 'pilote', 'experience': 1}

again = build_fleet(data)
print(again.to_dict() == data)              # True
```

Indices

- Revoyez la diapo « Sauvegarder des objets » du cours 3.

- Dans `Operator` , ne réécrivez pas les quatre clés de `Person` : partez de `super().to_dict()` puis complétez le dictionnaire.
- Dans `Spaceship` et `Fleet` , chaque objet contenu sait déjà se convertir : une boucle qui appelle leur `to_dict()` suffit. C'est la composition qui travaille pour vous.



Sauvegarder et charger

≈ 1 h

JSON · fichiers · exceptions

Dans un nouveau fichier `storage.py`, écrivez :

1. `save_fleet(fleet, path)`, qui enregistre `fleet.to_dict()` dans un fichier JSON **lisible** : indenté, en UTF-8, avec les accents conservés.
2. `load_fleet(path)`, qui relit le fichier et renvoie un objet `Fleet` grâce à `build_fleet()`.

`load_fleet()` ne doit **jamais faire planter le programme**. Quand le chargement est impossible, elle lève une `ValueError` avec un message clair, que le menu affichera :

Situation	Exception d'origine	Message conseillé
le fichier n'existe pas	<code>FileNotFoundError</code>	aucune sauvegarde « <code>fleet.json</code> »
le JSON est mal formé	<code>json.JSONDecodeError</code>	la sauvegarde « <code>fleet.json</code> » est corrompue
une clé attendue manque	<code>KeyError</code>	la sauvegarde « <code>fleet.json</code> » est incomplète
une valeur est interdite (rôle inconnu...)	<code>ValueError</code> de vos classes	le message de votre classe, tel quel

Résultats attendus

```
save_fleet(fleet, "fleet.json")
print(load_fleet("fleet.json").to_dict() == fleet.to_dict()) # True

load_fleet("missing.json")
# ValueError: aucune sauvegarde « missing.json »
```

Indices

- Revoyez les diapos « Écrire et lire du JSON » et « Charger sans planter » du cours 3.
- `json.dump(data, f, indent=2, ensure_ascii=False)` dans un fichier ouvert avec `encoding="utf-8"`.
- Dans un `except`, vous pouvez lever une nouvelle exception : `raise ValueError("...")`.



Le menu

≈ 30 min

Menu · exceptions

1. Ajoutez [7] **Sauvegarder la flotte** et [8] **Charger une flotte**.
2. Le nom du fichier est demandé à l'utilisateur, avec `fleet.json` par défaut s'il appuie simplement sur Entrée.
3. Si le chargement échoue, le message d'erreur s'affiche et **la flotte actuelle est conservée**.



Exemple de session

```
Votre choix : 7
Fichier de sauvegarde [fleet.json] :
Flotte Galactica sauvegardée dans « fleet.json ».

Votre choix : 8
Fichier à charger [fleet.json] : old.json
Chargement impossible : aucune sauvegarde « old.json »
```

4

Testez comme un attaquant

≈ 30 min

Validation des entrées

Sécurité

Un fichier de sauvegarde se modifie avec n'importe quel éditeur de texte. Mettez-vous à la place d'un utilisateur malveillant : faites une copie de `fleet.json`, modifiez-la, puis essayez de la charger. Votre programme doit réagir ainsi :

Modification dans la copie	Comportement attendu
un rôle devient <code>"root"</code>	chargement refusé : <code>rôle inconnu : root</code>
un mana devient <code>9999</code>	chargé, mais le mana est ramené à 100 par votre setter
la clé <code>"age"</code> d'un membre est supprimée	chargement refusé : sauvegarde incomplète
la dernière accolade <code>}</code> est supprimée	chargement refusé : sauvegarde corrompue

Réfléchissez

- Pour le mana, votre programme **corrige** la valeur ; pour le rôle, il la **refuse**. Quelle stratégie est la plus sûre ? Dans quels cas ?
- Si l'attaquant change un âge de 37 en 38, aucune de vos validations ne le voit : la valeur est plausible. Pour détecter **toute** modification, il faut une empreinte de la sauvegarde : c'est le bonus 5.
- C'est pour éviter ces pièges que l'on utilise JSON et jamais `pickle` ni `eval()` sur des données non fiables (cours 3).

5

Fichiers de test et README

≈ 15 min

Git · documentation

1. Commitez **deux fichiers de test** dans votre dépôt : une sauvegarde valide (`fleet.json`) et une sauvegarde piégée (`fleet-corrupted.json`), que votre programme doit refuser proprement.
2. Complétez votre `README.md` :
 - comment lancer le programme et à quoi sert chaque option du menu ;
 - les bonus réalisés, TP par TP ;
 - votre **déclaration d'usage de l'IA** : quels outils, pour quelles tâches.



Pour aller plus loin

Facultatif

Bonus 1 – Événements aléatoires

- Après chaque action du menu, un événement peut survenir au hasard : une **attaque du Mulet** endommage un vaisseau opérationnel. Ou bien des **renforts** rejoignent un vaisseau qui a encore de la place.
- Utilisez le module `random` : il convient parfaitement pour un jeu. Rappel du cours 1 : jamais pour générer un secret !

Bonus 2 – Statistiques enrichies

- Complétez l'option [6] avec le nombre de vaisseaux opérationnels et endommagés.

Bonus 3 – Chargement automatique

- Au lancement, si `fleet.json` existe, la dernière sauvegarde est chargée automatiquement. Sinon, on part de `FLEET_DATA`.

Bonus 4 – Parcours confirmé : `from_dict()`

- Remplacez `build_fleet()` par une méthode de classe `from_dict()` dans chaque classe, symétrique de `to_dict()`. Découvrez le décorateur `@classmethod`.

Bonus 5 – Protéger la sauvegarde par une empreinte

- À chaque sauvegarde, calculez l'**empreinte SHA-256** de `fleet.json` (cours 3) et enregistrez-la dans `fleet.json.sha256`.
- Au chargement, recalculez l'empreinte : si elle diffère, refusez le fichier. Même le passage d'un âge de 37 à 38 est alors détecté.
- Réfléchissez : un attaquant qui peut modifier `fleet.json` peut aussi recalculer l'empreinte. Comment l'en empêcher ?



Avant de rendre

Vérifiez chaque point. Ils correspondent à la grille d'évaluation du projet Fondation.

- ☐ Chaque classe a sa méthode `to_dict()` et l'aller-retour `build_fleet(fleet.to_dict())` est parfait.
- ☐ La sauvegarde est un JSON lisible : indenté, accents conservés.
- ☐ Après une sauvegarde, un redémarrage puis un chargement, la flotte est identique (membres, expérience, mana, état des vaisseaux).
- ☐ Les quatre attaques de l'étape 4 et le fichier absent sont gérés sans jamais faire planter le programme.
- ☐ Les options [0] à [8] du menu fonctionnent.
- ☐ Le dépôt contient `fleet.json`, `fleet-corrupted.json` et un `README.md` complet, avec la déclaration d'usage de l'IA.
- ☐ Des commits réguliers ont été poussés, avec des messages clairs.
- ☐ Vous pouvez expliquer **chaque ligne** de votre code.

Et le 5 novembre ?

Vous aurez encore du temps en séance pour terminer le TP3. Le projet complet est à rendre **au plus tard à 16h30**, juste avant l'examen écrit. Pendant la journée, je vous recevrai un par un pour la soutenance. Arrivez avec un projet qui fonctionne !